

Measuring Decidability as Related to Busy Beaver Numbers

Gurpreet Tandi
Bakersfield College
tandi.sgurpreet@gmail.com

Josue Gonzalez-Hendrix
Bakersfield College
jgonzalezhendrix@gmail.com

Dr. Jonathan P. Brown, PhD
Bakersfield College
jonathan.brown@bakersfieldcollege.edu

May 15, 2026

Abstract

The theoretical existence of Busy Beaver numbers provides a new notion for decidability and corresponding heuristics for conjectures. The minimum number of states in which a conjecture can be modeled gives a classification of what logic system can describe said conjecture. In this work, we construct explicit Turing machines that search for a solution to Brocard's problem greater than $n = 7$ and a Fermat prime beyond $n = 4$ which halt if and only if such solutions exist.

1 Introduction

1.1 Background and Motivation

Brocard's problem was first posed in 1876 and the hunt for Fermat primes began in around 1650[2, 4]. The decidability of whether or not unknown solutions exist for Brocard's problem is unknown along with the decidability of whether or not unknown Fermat primes exist. In this paper, we provide upper bounds for the number of states TMs (TM) searching for such numbers can be composed of by describing a TM which halts if and only if it finds an unknown solution to Brocard's problem and another which halts if and only if it finds an unknown Fermat prime.

There exist two widely accepted notions of decidability: a problem is decidable if a TM that always halts in finite time can be constructed which returns a yes or no for a given input (known as 'TM decidable'), or if there exists an algorithmic procedure for determining whether a statement is true or false in finitely many steps.

We may use Busy Beaver numbers to circumvent the requirement of a TM always halting if the TM is designed to halt if and only if a statement is true which means that if the TM does not halt then the statement is false. Furthermore, we associate TMs with logic systems, by the number of states a TM contains corresponding to the strength of the logic system. That is, if the smallest TM which describes a conjecture is smaller than one which describes a different conjecture, the first conjecture requires a weaker logic system. In this way, we

may describe the conjecture requiring a weaker logic system as having a higher decidability than the other conjecture.

1.2 Turing Machines

While many different types of TMs, which are all able to emulate one another, exist, we use the definition of a TM relevant to busy beaver numbers[5]. In this definition, there is one tape composed of an infinite number in both directions of sequential cells which initially contain zeroes and may contain either a one or a zero, and states read the current value of the current cell, write, move to an adjacent cell, then transition to another state, with the latter three actions depending on which value was read.

1.3 The Busy Beaver Function

The busy beaver number of N , $(BB(N))$, describes the maximum number state transitions a TM of size N that halts can take before halting[5]. For example, if a TM T is composed of four states, and runs for at least 108 steps, we know it will never halt because $BB(4) = 107$ [1].

The function $BB(N)$ is well defined. To see this, consider the set of TM of size N which always halt. There are finitely many TMs of size N , and so this subset is also finite. $BB(N)$ is just the maximum of steps required to halt on this set.

The Busy Beaver function is proven undecidable, otherwise the halting problem would be decidable [5]. This makes the the BB based notion of decidability a weaker notion than standard decidability. However, this loosening of conditions does induce an interesting heuristic for decidability that allows for categorization of conjectures.

2 A TM That Halts If and Only If It Finds an Unknown Fermat Prime

2.1 Description

The Fermat number of n , F_n , is 2^{2^n} . Since around the 1650's, we have known these to be prime for $n = 0, 1, 2, 3, 4$, but not any other numbers[4, 3]. It has been a long-standing conjecture that these are the only Fermat Primes.

This TM has three sections, described in appendix A, B, and C. The first section computes 2^{16} given a tape populated only by zeros, the second section squares the number on the tape and adds one to it, and the third section halts if the number on the tape is prime and decrements the number by one if it is composite and calls the second section of the machine. The numbers recorded on the tape are always in unary. By combining states which will only ever read a one or a zero into states which have behaviors for both read cases, we find that the upper bound for the number of states a TM which halts if and only if at least one unknown Fermat prime exists is 72 states.

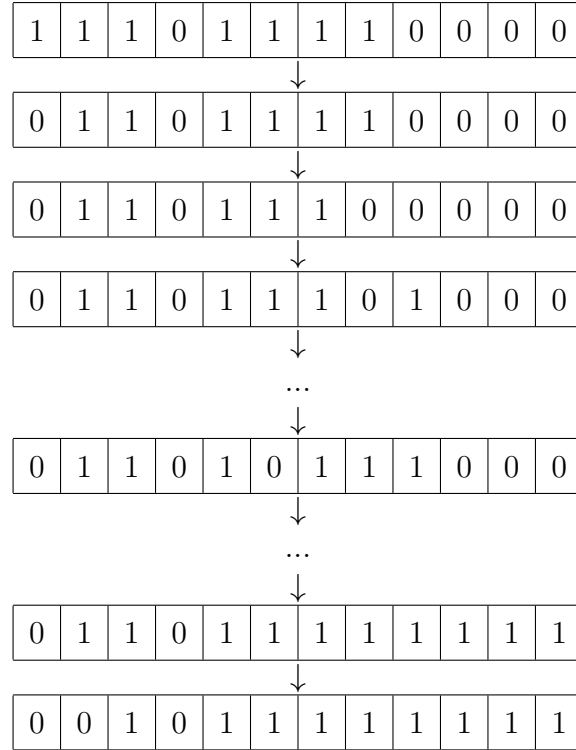
2.2 Explanation

2.2.1 Initialization

The first section of the machine computes the largest power of two corresponding to a known Fermat prime, the fourth Fermat number minus 1.

$$\begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hline \end{array} \rightarrow \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 \\ \hline \end{array}$$

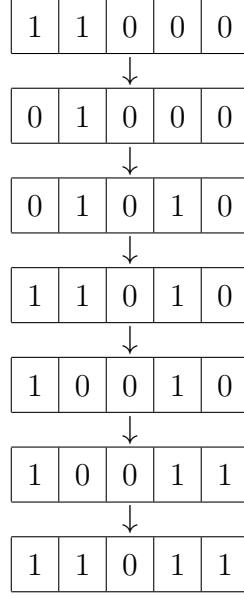
States 25-34 on Table 2 write four and three in unary onto the blank tape, leaving the current cell as the leftmost cell shown above.



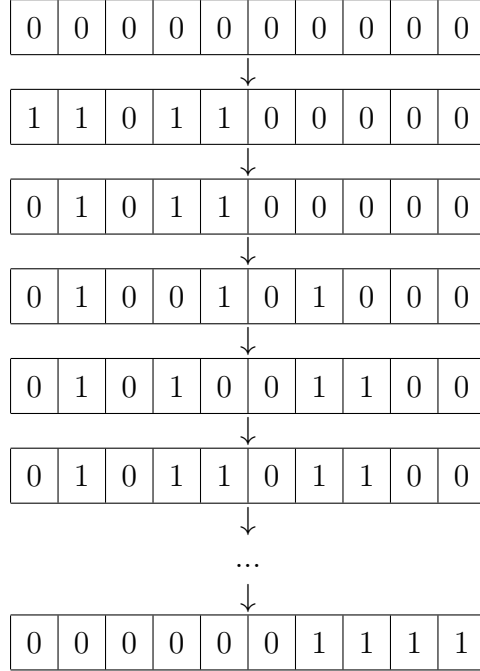
States 2-11 double the unary four three times by decrementing the unary three as an index, as shown above. If we call the number being doubled x and the index i , we can say that state 9 dictates whether x will be doubled by checking if i is equal to zero. When i is equal to zero, x is equal to 16. State 0 and states 12-14 set x equal to one and use the 32 as the index, allowing states 15-23 to double x i times as states 2-11 did. When state 15 detects i is equal to zero, the current cell is set to the leftmost cell of the current number and the next section of the TM is called.

2.2.2 Calculating the Next Fermat Number

We obtain the next Fermat number by squaring one less than previous Fermat number, and then adding one. Let x be the number being squared on the tape. States 0-4 on Table 2 skip a cell and copy x to the right of itself, as shown below.



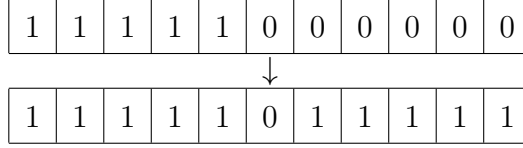
Now, we may square by multiplying the two unary numbers on the tape. Let the left number be called i and the right be called x . To obtain our desired output, x^2 , we multiply x by i and write the output to the right of x separated by a cell containing zero, as below.



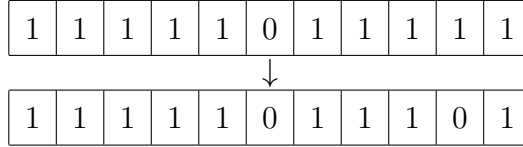
States 6-15 on Table 2 erase the index and output x 's square to the right of x . State 17 on Table 2 must exist in order to traverse cells, and it erases x and increments the square by one. Then, we have F_n and the next section of the TM is called.

2.2.3 Primality Check

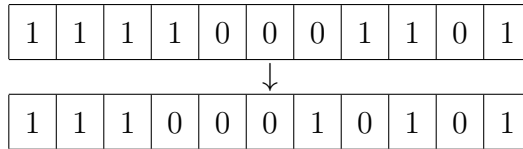
We now have F_n and must check for primality. If and only if F_n is prime, the machine halts. If the number is composite, it will be decremented by one and the first state of Table 2 is called.



Using an arbitrary number to demonstrate this algorithm, let's pretend the number we ended up with was five and call it x . States zero to four are a simple algorithm for duplicating x like above.

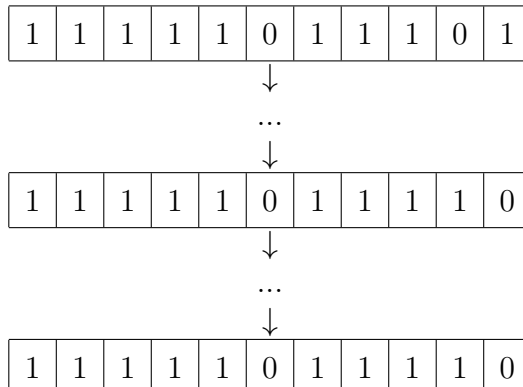


States 5-8 separate the duplicate of x into two indices: i , which is in the middle, and r , which is on the far right. This portion of the machine performs $x \% i$, decrementing r and incrementing i each time the module is not zero. When $r = 0$, the number is found to be prime. Note that i must be initialized as 3 for the machine to work properly and that this algorithm cannot test a number smaller than five.

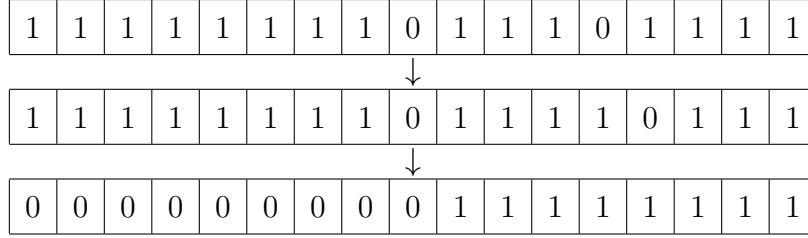


States 10-16 and 32-34 increment the index and decrement x by 3. States 12, 16, and 35 handle the cases in which x reaches zero, in which case x is restored to its original size, i is incremented and r is decremented. Since the final element of i being the index is a special case because if x reaches zero at that point then the number is composite, the last element isn't switched to a zero and states 36-39 handle the logic of decrementing x and deciding if the number is composite. If $x = 0$, state 38 calls state 43-46 and state 24 which repair the tape to solely include x and decrement x by one and call the first state of the square function.

There are three separate cases for this method of factoring due to the fact that the index can be found at the beginning of i , the state immediately after the beginning of i , or a state after those two. The logic is based on the machine "catching" itself during the third case by checking if the number after the number being indexed is zero to accommodate for factoring by a number greater than three. State 35 also dictates whether or not this correction is made. Below is an example of a number being found to be prime, the only halt case.



After x has been restored, the machine attempts to increment i and in doing so finds that $r = 0$, and therefore the machine halts. State specifically checks if $r = 0$. Below is an example of the machine finding a number composite.



The description of the nature of what the machine has done in this case is present earlier in the paper.

3 A TM That Halts If and Only If It Finds an Unknown Solution to Brocard's Problem

3.1 Description

Despite being explored since 1876, Brocard's problem $n! + 1 = m^2$, where m and n are integers, has no known solution for n beyond $n = 4, 5, 7$ where m and n are integers[2, 6]. It has long been conjectured that these are the only Brocard's numbers.

This TM two sections, denoted by 'F' states and 'S' states, both described in appendix D. The 'F' states compute a factorial recursively using an index and then adds one to it and the 'S' states check if the resulting number is a square by checking if it is the sum of consecutive odd numbers, in which case the machine halts. Combining the states which only have a 0 read behavior with the states that only have a 1 read, we find that an upper bound for the number of states a TM that halts if and only if at least one unknown solution to Brocard's problem exist is 43.

3.2 Explanation

The machine starts off in the "middle" of an infinitely long tape. The states Df, Jf, Pf, Qf, and Es are used here initialize the tape with (from left to right) n and $n!$ for $n = 2$, separated by a 0, and leaves the tape to the leftmost 0 of n . States Af, Bf, Cf, and Df increment n , then enter $(n - 1)!$.

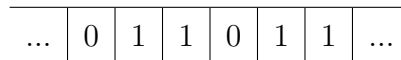


Figure 1: The tape after the first five steps

States Ef, Ff, Gf, Hf If, and Jf adds $(n - 1) * (n - 1)!$ to a new sequence of 1s to the right of $(n - 1)!$, separated by a 0. Once it finishes, Hf, Kf, and Lf check to see for completion, transitioning to Af if the calculation is finished. Once it has, the head returns to n , then

Mf, Nf, Of, Pf, Qf, Rf, and Sf test if $n > 7$. If any of these states encounters a 1, Vf, Wf, Xf and Sf restart the factorial calculation. If none of these states reads a 1, Tf and Uf go to the right of $(n - 1)!$ and $(n - 1) * (n - 1)!$, changing the separating 0 to a 1, leaving the value as $n! + 1$, transitioning into As, the beginning of the square checking half of the machine.

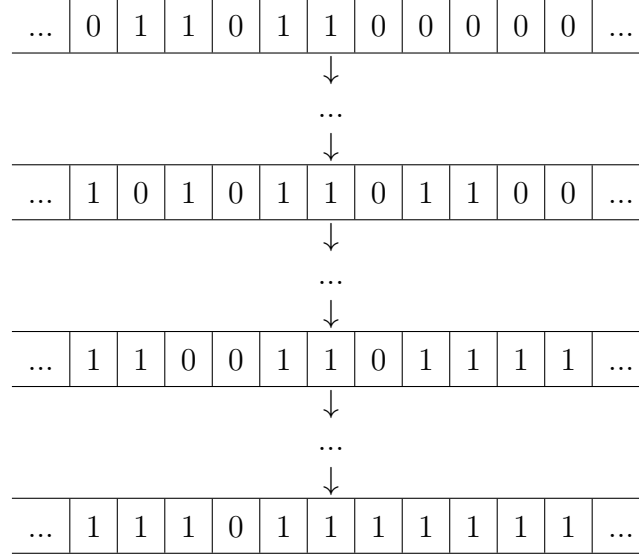


Figure 2: The tape during the calculation of $3!+1$

To check if $n! + 1$ is a square, the machine recursively "subtracts" odd numbers, starting with 1, from $n! + 1$, replacing 1s with 0s. For an integer $2k + 1$, it first subtracts $2k$ number of 0s, only subtracting the final $+1$ if it hasn't subtracted away all of $n! + 1$.

It keeps track of the current k with a sequence of 1s to the right of $n! + 1$. Af changes the 0 to the right of $n! + 1$ to a 1 that will never be subtracted so it doesn't lose track of where the $n! + 1$ is and goes right. Bs goes to the right, leaving the 0 that will separate the $n! + 1$ from k .

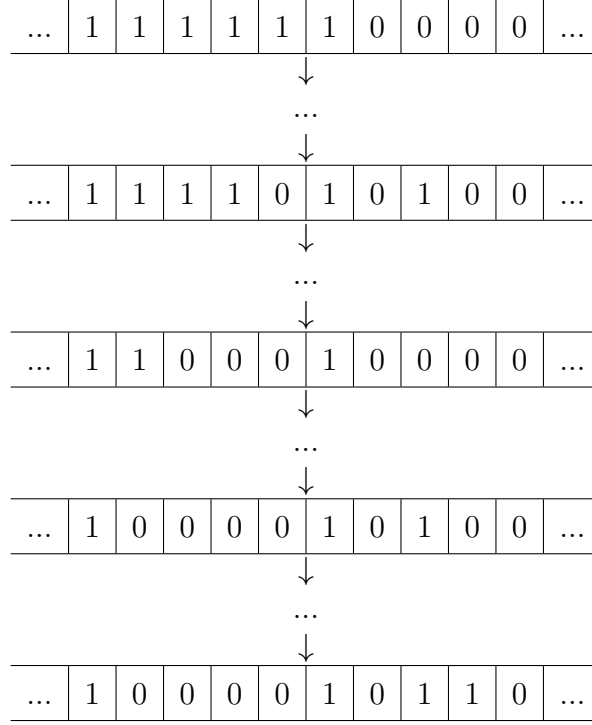


Figure 3: The tape while “subtracting” $2k + 1$ from $n! + 1$ for $k = 0$, $k = 1$, then assigning $k := 2$

Cs checks if it has finished subtracting $2k$, and if it hasn’t, Ds, Es, Fs, Gs, Hs, Is, and Js subtract 2. If the machine reaches the end of $n! + 1$ during this process, then Os, Ps, Qs, Rs, and Ss undo the subtraction, remove k , the separating 1, and the right-most 1 of $n! + 1$ before transitioning into Kf to calculate the next factorial. If Cs didn’t read a 0, then the machine managed to finish subtracting $2k$, so Ls and Ms subtract the final $+1$. Ns checks if there is anything left over. If there isn’t, then $n! + 1$ is of the form

$$\sum_{i=0}^k 2k + 1$$

for some k and is a square, so the machine halts. If there is still more, then Ks returns to k and Bs increments it, allowing it to subtract the next odd integer.

4 Related Works

Adam Yedida and Scott Aaronson’s work *A Relatively Small TM Whose Behavior Is Independent of Set Theory* inspired us in writing about TMs and logic systems by showing that it is possible to construct TMs which halt if and only if a logic system is inconsistent[7].

References

- [1] A. H. Brady. The determination of the value of rado’s noncomputable function $\sigma(k)$ for four-state turing machines. *Mathematics of Computation*, 40(162):647–665, 1983.
- [2] H. Brocard. Question 166. *Nouvelle correspondance mathématique*, 2:287, 1876.
- [3] Richard Crandall and Carl Pomerance. *Prime Numbers: A Computational Perspective*. Springer, 2 edition, 2005.
- [4] L. Euler. Observationes de theoremate quodam fermatiano aliisque ad numeros primos spectantibus. *Commentarii Academiae Scientiarum Petropolitanae*, 6:103–107, 1738.
- [5] T. Rado. On non-computable functions. *Bell System Technical Journal*, 41:877–884, 1962.
- [6] Srinivasa Ramanujan. Question 469. *Journal of the Indian Mathematical Society*, 5:59, 1913.
- [7] Adam Yedidia and Scott Aaronson. A relatively small turing machine whose behavior is independent of set theory, 2016.

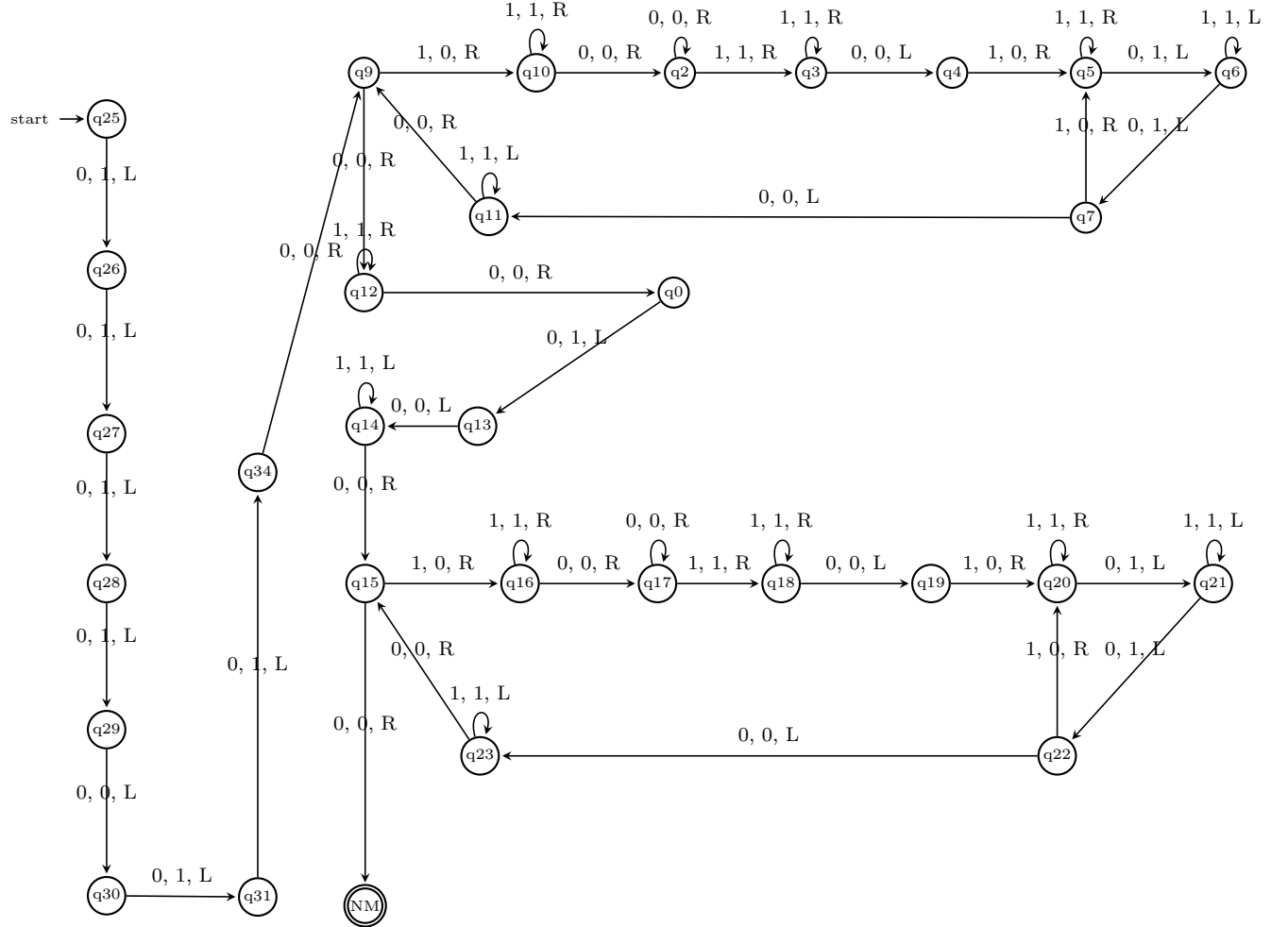
Appendices

A FP Table 1 and Visualization

State	Reads	Writes	Moves	Calls
0	0	1	L	13
2	1	1	R	3
2	0	0	R	2
3	0	0	L	4
3	1	1	R	3
4	1	0	R	5
5	0	1	L	6
5	1	1	R	5
6	0	1	L	7
6	1	1	L	6
7	0	0	L	11
7	1	0	R	5
9	0	0	R	12

9	1	0	R	10
10	0	0	R	2
10	1	1	R	10
11	0	0	R	9
11	1	1	L	11
12	0	0	R	0
12	1	1	R	12
13	0	0	L	14
14	0	0	R	15
14	1	1	L	14
15	0	0	R	NM
15	1	0	R	16
16	0	0	R	17
16	1	1	R	16
17	1	1	R	18
17	0	0	R	17
18	1	1	R	18
18	0	0	L	19
19	1	0	R	20
20	0	1	L	21
20	1	1	R	20
21	1	1	L	21
21	0	1	L	22
22	0	0	L	23
22	1	0	R	20
23	0	0	R	15
23	1	1	L	23
25	0	1	L	26
26	0	1	L	27
27	0	1	L	28

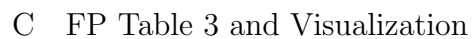
28	0	1	L	29
29	0	0	L	30
30	0	1	L	31
31	0	1	L	31
34	0	0	R	9



B FP Table 2 and Visualization

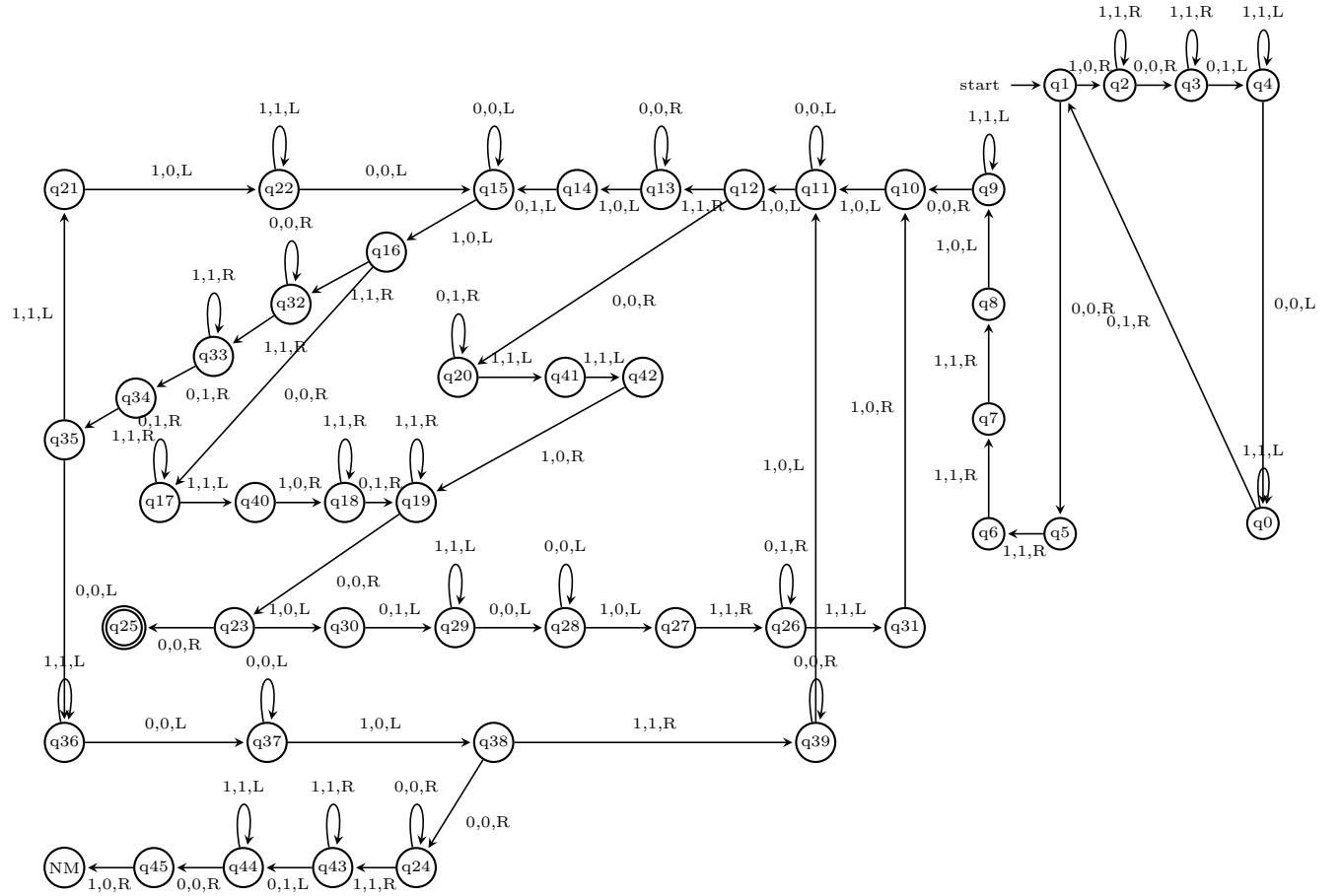
State	Reads	Writes	Moves	Calls
0	0	0	L	5
0	1	0	R	1
1	1	1	R	1
1	0	0	R	2

2	0	1	L	3
2	1	1	R	2
3	0	0	L	4
3	1	1	L	3
4	0	1	R	0
4	1	1	L	4
5	0	0	R	6
5	1	1	L	5
6	0	0	R	17
6	1	0	R	7
7	0	0	R	8
7	1	1	R	7
8	1	0	R	9
9	0	0	R	10
9	1	1	R	9
10	0	1	L	11
10	1	1	R	10
11	0	0	L	12
11	1	1	L	11
12	1	1	L	13
12	0	1	L	14
13	1	1	L	13
13	0	1	R	8
14	0	0	L	15
14	1	1	L	14
15	0	0	R	6
15	1	1	L	15
17	0	1	R	NM
17	1	0	R	17

13

13	0	0	R	13
14	0	1	L	15
15	1	0	L	16
15	0	0	L	15
16	0	0	R	17
16	1	1	R	32
17	0	1	R	17
17	1	1	L	40
18	0	1	R	19
18	1	1	R	18
19	0	0	R	23
19	1	1	R	19
20	1	1	L	41
20	0	1	R	20
21	1	0	L	22
22	0	0	L	15
22	1	1	L	22
23	1	0	L	30
23	0	0	R	25
24	1	1	R	43
24	0	0	R	24
26	1	1	L	31
26	0	1	R	26
27	1	1	R	26
28	1	0	L	27
28	0	0	L	28
29	1	1	L	29
29	0	0	L	28
30	0	1	L	29
31	1	0	R	10

32	1	1	R	33
32	0	0	R	32
33	0	1	R	34
33	1	1	R	33
34	1	1	R	35
35	0	0	L	36
35	1	1	L	21
36	0	0	L	37
36	1	1	L	36
37	1	0	L	38
37	0	0	L	37
38	1	1	R	39
38	0	0	R	24
39	1	0	L	11
39	0	0	R	39
40	1	0	R	18
41	1	1	L	42
42	1	0	R	19
43	0	1	L	44
43	1	1	R	43
44	0	0	R	45
44	1	1	L	44
45	1	0	R	NM



D BP Table and Visualization

State	Reads	Writes	Moves	Calls
Af	0	1	R	Bf
Af	1	1	L	Af
Gf	0	0	L	Hf
Gf	1	1	L	Gf
Hf	1	1	L	If
Hf	0	1	L	Kf
If	0	1	R	Jf
If	1	1	L	If
Cf	0	0	R	Df
Cf	1	1	R	Cf
Jf	1	0	R	Ef

Kf	0	0	L	Lf
Kf	1	1	L	Kf
Lf	0	1	L	Mf
Lf	1	1	L	Af
Pf	0	0	R	Vf
Pf	1	1	L	Qf
Ff	0	1	L	Gf
Ff	1	1	R	Ff
Qf	1	1	L	Rf
Qf	0	0	R	Vf
Mf	1	1	L	Nf
Nf	1	1	L	Of
Of	1	1	L	Pf
Of	0	0	R	Vf
Rf	0	0	R	Vf
Rf	1	1	L	Sf
Sf	1	1	R	Tf
Sf	0	0	R	Vf
Df	1	0	R	Ef
Tf	0	0	R	Uf
Tf	1	1	R	Tf
Ef	0	0	R	Ff
Ef	1	1	R	Ef
Bf	1	0	R	Cf
Uf	0	1	R	As
Uf	1	1	R	Uf
Vf	0	0	R	Wf
Vf	1	1	R	Vf
Wf	0	1	R	Xf
Wf	1	1	R	Wf

R _s	1	0	R	Q _s
R _s	0	0	L	S _s
Q _s	0	0	R	R _s
Q _s	1	0	R	Q _s
X _f	1	1	R	X _f
X _f	0	0	L	S _s
P _s	0	0	R	Q _s
O _s	1	0	R	P _s
O _s	0	1	R	O _s
N _s	0	1	L	HALT
N _s	1	1	R	K _s
M _s	1	0	L	N _s
M _s	0	0	L	M _s
L _s	0	0	L	L _s
L _s	1	1	L	M _s
K _s	1	1	R	B _s
K _s	0	0	R	K _s
J _s	0	0	R	O _s
J _s	1	1	R	K _s
I _s	1	0	L	J _s
I _s	0	0	R	O _s
H _s	1	0	L	I _s
H _s	0	0	L	H _s
G _s	1	1	L	H _s
F _s	0	0	L	G _s
F _s	1	1	L	F _s
E _s	1	0	L	F _s
D _s	0	1	L	E _s
D _s	1	1	R	D _s
C _s	1	1	R	D _s

